

Architektura Silnika Przetwarzania Sygnałów DSP

Architektura i Implementacja Autorskiego Silnika Przetwarzania Sygnałów Cyfrowych (DSP)

1. Wprowadzenie do formatu RIFF i struktury plików WAV

Format RIFF (Resource Interchange File Format) to uniwersalny standard zapisu danych multimedialnych, oparty na strukturze bloków danych (chunków). W kontekście cyfrowego dźwięku, najpopularniejszą implementacją tego kontenera jest format WAVE (WAV).

Z punktu widzenia inżynierii oprogramowania, plik WAV jest plikiem binarnym składającym się z dwóch głównych sekcji: 44-bajtowego nagłówka opisującego właściwości strumienia (metadane) oraz ciągłej tablicy zawierającej surowe dane cyfrowe (dane PCM lub zmiennoprzecinkowe IEEE 754). Poprawne zmapowanie nagłówka do struktury w języku C/C++ z wykorzystaniem kompilatorowej dyrektywy usuwającej wyrównywanie pamięci (np. `#pragma pack(push, 1)`) jest kluczowe do poprawnego odczytu i zapisu strumienia audio.

Struktura nagłówka RIFF/WAVE (44 bajty) Poniższa tabela przedstawia dokładny układ pamięci nagłówka wymaganego do prawidłowej obsługi plików audio.

Nazwa Pola	Typ Danych w C++	Rozmiar (Bajty)	Rola w strukturze pliku
ChunkID	char[4]	4	Sygnatura głównego kontenera. Zawsze zawiera znaki "RIFF".
ChunkSize	uint32_t	4	Rozmiar reszty pliku wyrażony w bajtach.
Format	char[4]	4	Sygnatura formatu. Zawsze zawiera znaki "WAVE".
Subchunk1ID	char[4]	4	Sygnatura sekcji formatowania. Zawsze zawiera znaki "fmt ".

Nazwa Pola	Typ Danych w C++	Rozmiar (Bajty)	Rola w strukturze pliku
Subchunk1Size	uint32_t	4	Rozmiar sekcji formatu. Wynosi 16 dla podstawowego formatu PCM.
AudioFormat	uint16_t	2	Kod formatu danych (1 = nieskompresowane PCM, 3 = zmiennoprzecinkowe Float).
NumChannels	uint16_t	2	Liczba kanałów (1 = Mono, 2 = Stereo).
SampleRate	uint32_t	4	Częstotliwość próbkowania (np. 44100 Hz), czyli ilość próbek na sekundę.
ByteRate	uint32_t	4	Przepustowość danych (ilość bajtów przetwarzanych w ciągu sekundy).
BlockAlign	uint16_t	2	Rozmiar pojedynczej ramki dla wszystkich kanałów.
BitsPerSample	uint16_t	2	Głębokość bitowa pojedynczej próbki (np. 16, 24, 32 bity).
Subchunk2ID	char[4]	4	Sygnatura sekcji danych. Zawsze zawiera znaki "data".
Subchunk2Size	uint32_t	4	Rozmiar samej zawartości audio (tylko surowe próbki w bajtach).

2. Szczegółowy opis zaimplementowanych efektów audio

2.1. Gain (Wzmocnienie Sygnału)

Teoria i zasada działania: Najprostsza operacja DSP polegająca na bezpośredniej zmianie amplitudy fali dźwiękowej. Realizuje się ją poprzez pomnożenie każdej próbki w buforze przez stałą wartość (mnożnik). Mnożnik o wartości 1.0 zostawia sygnał bez zmian. Mnożnik mniejszy niż 1.0 (np. 0.5) ścisza sygnał. Mnożnik większy niż 1.0 (np. 2.0) podgłasza sygnał, co niesie ryzyko przekroczenia bezpiecznej granicy cyfrowego zera (1.0 dBFS).

Algorytm (Krok po kroku):

1. Pobierz pojedynczą próbkę ze strumienia wejściowego.
2. Przeprowadź mnożenie wartości próbki przez zadany parametr wzmocnienia liniowego.
3. Nadpisz oryginalną próbkę w buforze wynikiem operacji i przejdź do następnej iteracji.

2.2. Distortion (Hard Clipping)

Teoria i zasada działania: Agresywne zniekształcenie sygnału, które generuje tony harmoniczne. Polega na celowym, bardzo mocnym podbiciu głośności (Drive) tak, aby fala dźwiękowa uderzyła w górny i dolny limit cyfrowego świata (od -1.0 do 1.0). Wszelkie wartości wystające poza ten próg są brutalnie "ucinane" na płasko, co zamienia okrągłą falę w falę zbliżoną do prostokąta. Odcięcie musi być bezwzględnie symetryczne, aby uniknąć przesunięcia składowej stałej (DC Offset).

Algorytm (Krok po kroku):

1. Odczytaj aktualną próbkę z bufora.
2. Wzmocnij próbkę, mnożąc ją przez współczynnik przesterowania.
3. Sprawdź, czy nowa wartość wykracza poza bezpieczny zakres. Jeśli przekracza 1.0, zablokuj ją na sztywno na 1.0. Jeśli spada poniżej -1.0, zablokuj na -1.0.
4. Zapisz spłaszczoną próbkę w wektorze.

2.3. Delay (Echo ze Sprzężeniem Zwrotnym)

Teoria i zasada działania: Efekt oparty na czasie. Zjawisko echa uzyskuje się poprzez odczytanie próbki z przeszłości (z określonym opóźnieniem) i dodanie jej do próbki aktualnej. Ponieważ algorytm nadpisuje główny bufor, odczytywanie przeszłości po pewnym czasie skutkuje odczytaniem "echa z echa", co tworzy naturalnie wygasający ogon (Feedback). Procesor nie rozumie milisekund, operuje na odległości w tablicy (indeksach). Przed pętlą główną rozmiar wektora musi zostać sztucznie powiększony (np. o wielokrotność offsetu), a nowe pozycje wypełnione cyfrowym zerem (ciszą). W przeciwnym razie ogon wybrzmienia zostanie brutalnie ucięty równo z oryginalnym czasem trwania piosenki.

Algorytm (Krok po kroku):

1. Przelicz podany czas w milisekundach na cyfrowy dystans w indeksach (offset).
2. Rozszerz rozmiar wektora wyjściowego i wypełnij nowe indeksy ciszą.
3. W pętli oblicz indeks próbki z przeszłości (obecny indeks minus offset).
4. Do aktualnej próbki dodaj wartość próbki z przeszłości przemnożoną przez parametr sprzężenia zwrotnego, a wynik zapisz w buforze.

2.4. Low-Pass Filter (Filtr IIR 1. rzędu)

Teoria i zasada działania: Filtr dolnoprzepustowy (odcinający wysokie tony, np. hi-haty) wygładza bardzo gwałtowne skoki wartości między sąsiadującymi próbkami. Działa na zasadzie uśredniania aktualnej próbki z bezpośrednią historią próbki poprzedniej (IIR - Infinite Impulse Response). W plikach stereo kanał lewy i prawy stanowią odrębne strumienie. Klasa efektu musi posiadać dwie zupełnie niezależne zmienne do przetrzymywania historii (jedną dla lewego, drugą dla prawego kanału) i wybierać odpowiednią w pętli za pomocą operacji modulo.

Algorytm (Krok po kroku):

1. Odczytaj aktualną próbkę audio.
2. Ustal kanał (lewy lub prawy) poprzez zbadanie parzystości indeksu.
3. Przemnóż obecną próbkę przez przepustowość filtra, a zapamiętaną próbkę historyczną przez dopełnienie matematyczne.
4. Zsumuj oba wyniki.
5. Zapisz zsumowany wynik w buforze oraz zaktualizuj odpowiednią zmienną historyczną.

2.5. Kompresor Dynamiki (Architektura Jednopasmowa)

Teoria i zasada działania: Cyfrowy kompresor nie działa jak proste ucinanie fali (clipper/saturator). Składa się z dwóch niezależnych bloków logicznych: statycznego kalkulatora tłumienia oraz filtra wygładzającego w dziedzinie czasu. Zanim kompresor cokolwiek ściszy, musi "usłyszeć" sygnał i ustalić, jak bardzo chce go zgnieść, co dzieje się w czasie zerowym. Algorytm wyciąga wartość bezwzględną z próbki, aby usłyszeć jej rzeczywistą energię, ignorując kierunek wychylenia głośnika, a następnie przelicza ją na decybele. Jeśli sygnał przekracza ustalony próg (Threshold), wyliczana jest nadwyżka (Overshoot), która jest dzielona przez proporcję (Ratio). Pozwala to wyliczyć Gain Reduction (GR), czyli dokładną ilość decybeli, o którą algorytm żąda ściszenia sygnału. Ten surowy rozkaz z powrotem przeliczany jest na liniowy ułamek (Target Multiplier).

Zastosowanie surowego Celu bezpośrednio na dźwięk spowodowałoby natychmiastowe zniekształcenie fali, dlatego kompresor musi aplikować zmianę płynnie. Wymaga to zmiennej zachowującej stan pomiędzy iteracjami pętli (pamięci operacyjnej kompresora), która zaczyna od wartości 1.0 i zawsze reprezentuje mnożnik z poprzedniej mikrosekundy. Czas realizowany jest poprzez nałożenie filtra dolnoprzepustowego IIR na sam mnożnik głośności, zmuszając go do płynnego poruszania się w stronę Celu. Wyliczane są dwie osobne wartości prędkości: dla Ataku (wpychanie suwaka w dół) oraz dla Release (puszczanie suwaka w górę). Jeśli Cel jest mniejszy niż pamięć, ładowana jest prędkość Ataku; jeśli większy, ładowana jest prędkość Release. Na samym końcu oryginalna próbka audio mnożona jest przez przeliczony, gładki mnożnik z pamięci stanu, dzięki czemu fala zachowuje swój kształt, a jedynie jej ogólna amplituda płynnie faluje.

Algorytm Kompresora (Krok po kroku):

1. **Nasłuch (Peak Detection):** Pobranie wartości bezwzględnej z próbki, aby usłyszeć faktyczną energię fali niezależnie od kierunku wychylenia głośnika.
2. **Tłumaczenie na dB:** Przeliczenie amplitudy sygnału ze skali liniowej na decybele.
3. **Bramka (Threshold):** Sprawdzenie, czy sygnał w decybelach jest głośniejszy niż ustalony próg.
4. **Kalkulacja Cięcia (Ratio):** Jeśli próg został przekroczony, wyliczana jest nadwyżka (Overshoot), a następnie redukcja (Gain Reduction) zgodnie z ustawioną proporcją.
5. **Powrót do Liniowości:** Zamiana wyliczonego tłumienia w decybelach z powrotem na docelowy mnożnik liniowy (od 0.0 do 1.0).
6. **Wybór Czasu:** Zestawienie docelowego mnożnika z historią. Jeśli docelowy mnożnik nakazuje ucięcie sygnału mocniej niż w poprzednim kroku – aktywowany jest współczynnik Ataku. W

przeciwnym razie – Release.

7. **Wygładzanie (Filtr IIR):** Zaktualizowanie pamięci kompresora o jeden matematyczny krok w stronę celu.
8. **Aplikacja:** Przemnożenie oryginalnej próbki audio przez wygładzoną wartość z pamięci.

2.6. Kaskadowy Kompresor Wielopasmowy (Multiband Compressor)

Teoria i zasada działania: Wielopasmowe przetwarzanie dynamiki w tym silniku DSP opiera się na koncepcji kaskadowych filtrów komplementarnych. Zamiast równoległego stosowania skomplikowanych filtrów środkowoprzepustowych, system wykorzystuje odejmowanie sygnałów w dziedzinie czasu. Pozwala to na uniknięcie przesunięć fazowych na częstotliwościach podziału. Sygnał po rozdzieleniu i ponownym zsumowaniu (bez kompresji) daje idealnie płaską odpowiedź częstotliwościową (tzw. zjawisko perfect reconstruction). Każdy etap wydziela jedno pasmo za pomocą filtru dolnoprzepustowego (IIR), a następnie odejmuje je od sygnału wejściowego danego etapu, generując "resztę" częstotliwościową dla kolejnych sekcji.

Rozdział sygnału wejściowego na 5 pasm realizowany jest sekwencyjnie z użyciem 4 zwrotnic o rosnących częstotliwościach odcięcia. Główny sygnał przechodzi przez filtr Low-Pass #1, a różnica tworzy Resztę 1. Reszta 1 przechodzi przez Low-Pass #2, różnica wyznacza Resztę 2, i tak dalej aż do Pasma 5 (High), gdzie różnica z ostatniego etapu naturalnie izoluje najwyższe częstotliwości, tworząc gotowe pasmo bez konieczności dodatkowego filtrowania. Wyizolowane bufora trafiają do niezależnych instancji algorytmu Kompresora (opisanego w punkcie 2.5), umożliwiając odrębną kontrolę dynamiki dla każdego zakresu częstotliwości, a następnie są sumowane w jednej pętli wyjściowej.

Algorytm (Krok po kroku):

1. Skopiuj wektor oryginalny do bufora roboczego.
2. Przepuść bufor roboczy przez filtr Low-Pass (np. 100 Hz), aby wyizolować Pasmo 1 (Low).
3. Odejmij Pasmo 1 od oryginalnego sygnału, uzyskując "Resztę 1".
4. Poddaj "Resztę 1" kolejnemu filtrowi Low-Pass (np. 500 Hz), aby wyizolować Pasmo 2, a następnie odejmij je od "Reszty 1", aby uzyskać "Resztę 2".
5. Powtarzaj ten proces do momentu uzyskania 4 przefiltrowanych pasm i finalnej reszty, stanowiącej Pasmo 5.
6. Aplikuj niezależną kompresję na każdy z 5 wyizolowanych wektorów.
7. Zsumuj (dodaj do siebie próbka po próbce) wszystkie 5 wektorów z powrotem do bufora głównego.

2.7. Korektor Parametryczny (Equalizer - Biquad Peaking Filter)

Teoria i zasada działania: Korektor bazuje na filtrze drugiego rzędu (tzw. Biquad w postaci Direct Form I). W przeciwieństwie do filtru Low-Pass pierwszego rzędu, Biquad analizuje głębszą historię sygnału, co pozwala na generowanie zjawiska rezonansu i modelowanie precyzyjnych kształtów (np. podbijanie lub wycinanie konkretnej częstotliwości). Interfejs filtru sterowany jest trzema parametrami: częstotliwością centralną w hercach (F_c), wartością wzmocnienia lub cięcia w decybelach ($Gain$) oraz parametrem dobroci (Q), który determinuje szerokość i ostrość krzywej filtra. Proces wymaga niezależnego zapamiętywania dwóch poprzednich klatek wejściowych oraz dwóch wyjściowych osobno dla obu kanałów.

Algorytm (Krok po kroku):

1. Podczas inicjalizacji w konstruktorze przelicz docelowe parametry ludzkie na radiany, przepustowość i amplitudę liniową.
2. Wylicz pięć współczynników dla bezpośredniego równania różnicowego, normalizując każdy z nich

(dzieląc przez a_0).

3. W głównej pętli zapisz surowy, oryginalny dźwięk do tymczasowej zmiennej pomocniczej.
4. Uruchom główne równanie Direct Form I korzystając z wejścia i zmiennych historycznych, nadpisując bufor wyjściowy.
5. Uaktualnij zmienne historyczne w kolejności odwrotnej (przepisz zmienne z indeksu 1 pod indeks 2, a pod indeks 1 wstaw wartość ze zmiennej tymczasowej dla historii wejścia oraz świeżo wyliczoną klatkę dla historii wyjścia).

3. Kompendium Wzorów Matematycznych

Wszystkie wzory wykorzystane podczas projektowania kaskady wielopasmowej, kompresora dynamiki oraz filtrów zestawiono poniżej.

3.1. Przekształcenia Skalarne i Formatowanie Czasu

Tłumaczenie skali liniowej na decybelową: Wykorzystywane między innymi w bloku detekcji kompresora.

$$dB = 20.0 \cdot \log_{10}(|x[n]|)$$

Tłumaczenie docelowego tłumienia kompresora z decybeli na liniowy ułamek (Target Multiplier):

$$TargetMultiplier = 10^{20.0/GR}$$

Obliczanie przesunięcia czasu w wektorze dla efektu Delay:

$$Offset = (1000.0 / SampleRate) \cdot Time_{ms} \cdot C$$

Zamiana zadanego wzmocnienia w decybelach na mnożnik liniowy (np. dla EQ):

$$A = 10^{\text{gain}/40}$$

3.2. Proste Równania Różnicowe (Filtry I. Rzędu)

Zjawisko echa ze sprzężeniem zwrotnym:

$$y[n] = x[n] + (x[n - Offset] \cdot Feedback)$$

Filtr dolnoprzepustowy (Low-Pass IIR): Równanie używane w kaskadzie zwrotnic.

$$y[n] = (\alpha \cdot x[n]) + ((1 - \alpha) \cdot y[n - 1])$$

Równanie uśredniające dla wewnętrznej pamięci kompresora: Filtr czasu modyfikujący mnożnik głośności zamiast samej fali.

$$Mnoznik = \alpha \cdot mnoznik + (1 - \alpha) \cdot TargetMultiplier$$

Wyliczanie współczynnika wygładzania (α) dla stałych czasowych kompresora:

$$\alpha = \exp(-1000.0 / \text{SampleRate} \cdot \text{Time}_{ms})$$

3.3. Algorytm Statyczny Kompresji Dynamiki

Wzory wyliczające bezwzględną wielkość cięcia (Gain Reduction) dla sygnałów przekraczających ustalony próg.

$$\text{Overshoot} = \text{dB} - \text{Threshold}$$

$$\text{GR} = \text{Overshoot} - (\text{Ratio} / \text{Overshoot})$$

3.4. Równania Filtru Korektora (Biquad Peaking EQ)

Zmienne pomocnicze determinujące charakterystykę filtra:

$$\omega_0 = 2 \cdot \pi \cdot F_c / \text{SampleRate}$$

$$\alpha = \sin(\omega_0) / (2 \cdot Q)$$

Obliczanie surowych współczynników dla filtra parametrycznego:

$$b_0 = 1 + \alpha \cdot A$$

$$b_1 = -2 \cdot \cos(\omega_0)$$

$$b_2 = 1 - \alpha \cdot A$$

$$a_0 = 1 + \alpha / A$$

$$a_1 = -2 \cdot \cos(\omega_0)$$

$$a_2 = 1 - \alpha / A$$

Równanie Różnicowe Biquad Direct Form I: Wzór uruchamiany na każdej próbce dźwięku (współczynniki muszą zostać wcześniej podzielone przez wartość a_0).

$$y[n] = (b_0 \cdot x[n]) + (b_1 \cdot x[n-1]) + (b_2 \cdot x[n-2]) - (a_1 \cdot y[n-1]) - (a_2 \cdot y[n-2])$$
